

牛客寒假营第2场题解

by wuuuk

本场答题数：9

A 一起奏响历史之音！

- 难度：签到
- 知识点：模拟

题意：

给定7个数，判断是不是只由12356中的部分或全部数字组成。

思路：

模拟即可。

代码：

```
a = list(map(int, input().split()))
b = [1, 2, 3, 5, 6]
sum = 0
for i in b:
    sum += a.count(i)
if sum == 7:
    print("YES")
else:
    print("NO")
```

B 能去你家蹭口饭吃吗

- 难度：签到
- 知识点：排序、中位数

题意：

给定数组 $\{a_n\}$ ，问不超过 $\{a_n\}$ 中一半元素的最大的数是多少。 $(n \leq 5 \cdot 10^5)$

思路:

排序之后，输出中位数减一。复杂度 $O(n \cdot \log n)$ 。

代码:

```
n = int(input())
a = list(map(int, input().split()))
a.sort()
ans = a[n // 2] - 1
print(ans)
```

F 一起找神秘的数!

- 难度：签到
- 知识点：位运算、打表

题意:

对于给定的区间 l 到 r ，从中选取两个整数 x 和 y ，使得下式成立：

$$x + y = (x \text{ or } y) + (x \text{ and } y) + (x \text{ xor } y)$$

求解有多少对不同的满足条件的 x, y 。

思路:

我们有（或者打表可知）：

$$x + y = (x \text{ or } y) + (x \text{ and } y)$$

因此题目所给条件成立当且仅当：

$$(x \text{ xor } y) = 0$$

即：

$$x = y$$

因此本题的答案就是区间长度。

代码:

```
T = int(input())
for t in range(T):
    l, r = [int(i) for i in input().split()]
    print(r - l + 1)
```

G 一起铸最好的剑!

- 难度: 签到
- 知识点: python语法、二分、模拟

题意:

牛可乐希望通过添柴使铸剑炉的温度最接近 n 度。初始温度为 m 度, 每次添柴温度变为原来的 m 倍。求最少需要添多少次柴 (包括初始的一次) 才能使温度最接近 n 。

思路:

由于指数增长, 次数不会超过32, 因此按照题意模拟即可, 当然也可以二分。

本题我想试试python里的对数函数。发现 $\log(n, m)$ 其实是 $\log_m n$ 。

另外, 如果使用对数计算, $m = 1$ 要特判。

代码:

```
from math import log, floor, ceil
T = int(input())
for t in range(T):
    n, m = [int(i) for i in input().split()]
    if (m == 1):
        print(1)
        continue
    tmp = log(n, m)
    guess1 = abs(pow(m, floor(tmp)) - n)
    guess2 = abs(pow(m, ceil(tmp)) - n)
    if guess1 <= guess2:
        print(max(floor(tmp), 1))
    else:
        print(max(ceil(tmp), 1))
```

J 数据时间?

- 难度: 签到
- 知识点: 模拟

题意:

给定一堆用户的id、登录日期和登录时间, 统计 h 年 m 月份用户在通勤、午休、临睡三个时段登录牛客 APP 的情况。每个用户的多次登录在同一时段内只计一次。

- 通勤时段: 07:00:00-09:00:00 和 18:00:00-20:00:00
- 午休时段: 11:00:00-13:00:00

- 临睡时段: 22:00:00-01:00:00

思路:

按照题意模拟即可，使用set存储usr_id来去重。可以用时间戳来判断时间是否属于某一时段。

代码:

```
n, h, m = [int(i) for i in input().split()]
tongqin = set() # 通勤
wuxiu = set() # 午休
linshui = set() # 临睡
def timestamp(hour, minute = 0, second = 0):
    return hour * 3600 + minute * 60 + second
for info in range(n):
    usr_id, login_date, login_time = [i for i in input().split()]
    year, month, day = [int(i) for i in login_date.split('-')]
    hour, minute, second = [int(i) for i in login_time.split(':')]
    if year == h and month == m:
        now = timestamp(hour, minute, second)
        if 0 <= now <= timestamp(1):
            linshui.add(usr_id)
        elif timestamp(7) <= now <= timestamp(9):
            tongqin.add(usr_id)
        elif timestamp(18) <= now <= timestamp(20):
            tongqin.add(usr_id)
        elif timestamp(11) <= now <= timestamp(13):
            wuxiu.add(usr_id)
        elif timestamp(22) <= now < timestamp(24):
            linshui.add(usr_id)
print(len(tongqin), len(wuxiu), len(linshui))
```

K 可以分开吗?

- 难度: 铜牌-
- 知识点: 搜索、并查集?

题意:

网格上有一些蓝色砖块和一些灰色砖块。只要敲除蓝色方块周围（上下左右）的灰色砖块蓝色砖块就会掉落。求掉落一个极大连通的蓝色砖块最少需要敲除多少灰色砖块。边界外不需要敲除。

思路:

和那个xpc一道数围棋棋子的气的题目一模一样。bfs跑一遍，使用set存储需要敲除的砖块即可。

复杂度 $O(n \cdot m \cdot \log(n \cdot m))$

代码:

```
#include <bits/stdc++.h>
#define rep(i, a, b) for (int i = a; i <= b; i++)
#define per(i, b, a) for (int i = b; i >= a; i--)
#define endl "\n"
using namespace std;
using pii = pair<int, int>;
constexpr int MAXN = 510;
int n, m;
char grid[MAXN][MAXN];
int walk[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
bool vis2[MAXN][MAXN], vis[MAXN][MAXN];
int bfs2(int sx, int sy) {
    queue<pii> q;
    vis2[sx][sy] = true;
    q.emplace(sx, sy);
    vis[sx][sy] = true;
    set<pii> need;
    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        for (auto [dx, dy] : walk) {
            int ux = x + dx;
            int uy = y + dy;
            if (ux < 1 or ux > n) continue;
            if (uy < 1 or uy > m) continue;
            if (vis[ux][uy]) continue;
            if (grid[ux][uy] == '0') {
                need.emplace(ux, uy);
            } else {
                vis[ux][uy] = true;
                q.emplace(ux, uy);
            }
        }
    }
    return need.size();
}
int main() {
    ios::sync_with_stdio(false), cin.tie(nullptr);
    cin >> n >> m;
    rep(i, 1, n) {
        rep(j, 1, m) {
```

```

        cin >> grid[i][j];
    }
}
int ans = n * m;
rep(i, 1, n) {
    rep(j, 1, m) {
        if (!vis[i][j] and grid[i][j] == '1') {
            ans = min(ans, bfs2(i, j));
        }
    }
}
cout << ans << endl;
return 0;
}

```

D 字符串里串

- 难度：铜牌-
- 知识点：观察、stl

题意：

字符串 s 的可爱度 k 为最大整数，使得存在长度为 k 的连续子串 a 和长度为 k 的不连续子序列 b ，满足 $a = b$ 。若不存在这样的 a, b ，则可爱度为 0。

- **连续子串**：从原字符串中连续选择一段字符得到的新字符串。
- **不连续子序列**：至少由两段不相邻的非空子串构成。

给定一个字符串 s ，求解其可爱度 k 。

思路：

注意到一旦有一个字母出现两次，那么就有一个符合题意的可爱字符串。若下标分别为 i 和 j ，则由这两个字母贡献的可爱度为： $\max\{i, n - j + 1\}$ 。我们把每个字母对应的下标使用 `map+vector` 存起来，然后对应查找每个字母的最大贡献即可。这个最大贡献应该是倒数第二个下标的前缀长度和第二个下标的后缀长度的最大值。复杂度 $O(n \cdot \log n)$ ，可以优化到 $O(n)$ （但没必要）。

代码：

```

#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false), cin.tie(nullptr);
    int n; cin >> n;
    string s; cin >> s;
    map<char, vector<int>>> mp;

```

```

for (int i = 0; i < n; i++) {
    char c = s[i];
    mp[c].emplace_back(i);
}
int ans = 0;
for (auto &[c, index] : mp) {
    int m = index.size();
    if (m < 2) {
        continue;
    }
    int i = index[m - 2];
    int j = index[1];
    ans = max(ans, i + 1);
    ans = max(ans, n - j);
}
// ans = 1 不满足定义，赛时不加这一行也能通过
if (ans == 1) ans = 0;
cout << ans << endl;
return 0;
}

```

C 字符串外串

- 难度：铜牌
- 知识点：构造、分类讨论

题意：

给定两个整数 n 和 m ，构造长度为 n ，可爱度为 m 的仅由小写字母组成的字符串。可爱度的定义见上一题。

思路：

首先，根据上一题的结论，可爱度所对应的字母（不妨让它是 'a'）必须在第 m 个位置或第 $n - m + 1$ 个位置。此时我们可以对这两个位置的先后顺序进行分类讨论。

1. 当 $m < n - m + 1$ ，即 $n \geq 2 \cdot m$ 时，

我们可以在这两个位置分别填上 'a'。此时，在 m 的左侧和 $n - m + 1$ 的右侧，这两端的内部不能出现相同字符，否则可爱度将大于 m ，但是互相之间的字符可以是一样的。在 m 到 $n - m + 1$ 之间，不能出现已有的字符，否则可爱度将大于 m 。

由于只有 26 个字母，我们有：

$$n - m + 1 - 1 \leq 26$$

即：

$$n - m \leq 26$$

这是存在可行解的必要条件。

2. 当 $m \geq n - m + 1$, 即 $n < 2 \cdot m$ 时,

此时我们同样可以在这两个位置分别填上'a', 不同的是, 在这两个位置中间我们也都都可以填'a', 因为这中间所贡献的可爱度不会超过 m 。在两端, 我们还是除了'a'外的其他字符, 两端内部不能有重复字符, 但两端互相可以有重复字符。

不难发现, 在这种情况下, 存在可行解的必要条件依然是 $n - m \leq 26$ 。

于是我们先根据必要条件判断是否有解, 然后根据 $2 \cdot m$ 与 n 的大小关系去分别构造即可。

复杂度 $O(n)$ 。

代码:

```
#include <bits/stdc++.h>
#define endl "\n"
using namespace std;
void print(const vector<char> &s) {
    for (auto c : s){
        cout << c;
    }
    cout << endl;
}
int main() {
    ios::sync_with_stdio(false), cin.tie(nullptr);
    int T; cin >> T;
    for (int ttt = 1; ttt <= T; ++ttt) {
        int n, m;
        cin >> n >> m;
        vector<char> a(n, '0');
        if (n - m > 26 or n == m) {
            cout << "NO" << endl;
        } else if (n >= 2 * m) {
            cout << "YES" << endl;
            a[m - 1] = 'a';
            a[n - m] = 'a';
            char c = 'b';
            for (int i = 0; i < m - 1; i++) {
                a[i] = a[n - i - 1] = c;
                c += 1;
            }
            for (int i = m; i < n - m; i++) {
                a[i] = c;
                c += 1;
            }
            print(a);
        } else {
            cout << "YES" << endl;
            for (int i = n - m - 1; i <= m; i++) {
```



```

        a[i] = 'a';
    }
    for (int i = 0; i < n - m - 1; i++) {
        a[i] = 'b' + i;
    }
    for (int i = m + 1, j = 0; i < n; i++, j++) {
        a[i] = 'b' + j;
    }
    print(a);
}
}
return 0;
}

```

H 一起画很大的圆！

- 难度：铜牌
- 知识点：几何、贪心

题意：

给定二维平面上的上下左右四个边界 $(-10^6 \leq a, b, c, d \leq 10^6)$ 所限定的矩形区域。要求在这个矩形区域边界上取三个整数坐标点，使得以三点为顶点的三角形外接圆半径最大。输出这三个点的坐标。

思路：

画一些图猜一下大概是哪些位置可以让半径尽量大，发现这题应该是有一个固定结论，不需要枚举。尝试的思路如下，

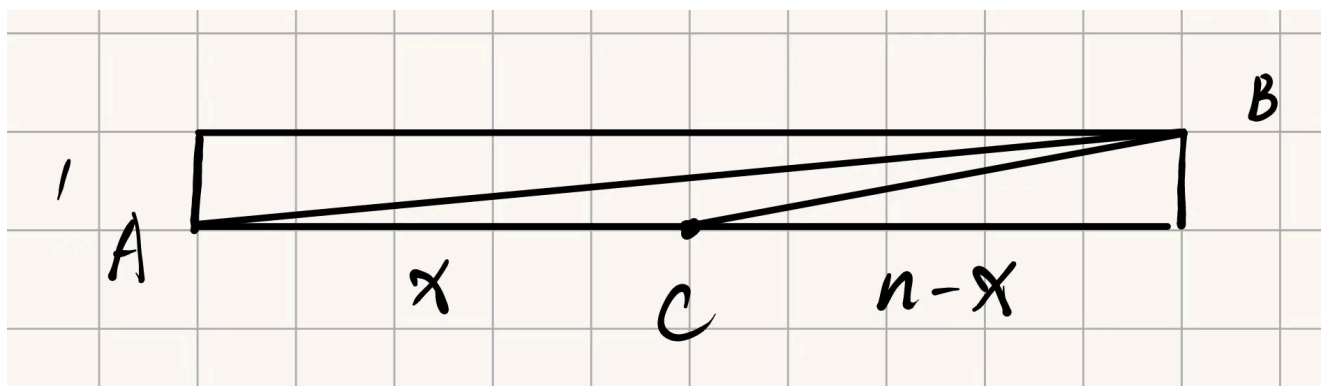
首先，根据正弦定理：

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$$

我们可以得到外接圆的半径公式：

$$R = \frac{abc}{4S_{\triangle ABC}}$$

对于一个宽度为 **1**，长度为 **n** 的矩形区域，我们很容易猜测对角线应该是一条边。我们固定对角线的两端为选定的两个点，然后可以看看第三个点要怎么选。



在上图中三角形ABC的外接圆半径为：

$$R = \frac{x \cdot \sqrt{n^2 + 1} \cdot \sqrt{(n-x)^2 + 1}}{4 \cdot \frac{x}{2}} = \frac{\sqrt{n^2 + 1} \cdot \sqrt{(n-x)^2 + 1}}{2}$$

当 $x = 1$ 时，取到最大值。

我们回到外接圆的半径公式，面积在分母上，其影响较大，因此让面积变小是首先应该考虑的，这与我们上面的推导一致。

接下来，考虑宽度为 m ，长度为 n 的情况。如果我们保持面积最小，即面积为0.5的话，让三边的长度乘积最大。此时要把三角形两条较长的边放在 n 和 m 中较大的位置上。猜测这样是外接圆半径最大的。提交之后，发现猜对了。

代码：

```
#include <bits/stdc++.h>
#define endl "\n"
using namespace std;
int main() {
    ios::sync_with_stdio(false), cin.tie(nullptr);
    int T; cin >> T;
    for (int ttt = 1; ttt <= T; ++ttt) {
        int a, b, c, d;
        cin >> a >> b >> c >> d;
        if (b - a > d - c) {
            cout << a << " " << c << endl;
            cout << a + 1 << " " << c << endl;
            cout << b << " " << c + 1 << endl;
        } else {
            cout << a << " " << c << endl;
            cout << a << " " << c + 1 << endl;
            cout << a + 1 << " " << d << endl;
        }
    }
    return 0;
}
```

E 一起走很长的路!

- 难度：铜牌+
- 知识点：前缀和、RMQ (ST表、线段树等)

题意：

有一个多米诺骨牌序列，给定 $\{a_n\}$ 为序列上每一个多米诺骨牌的重量。每次询问给出两个整数 l 和 r ，问如果推倒骨牌 l ，骨牌 r 会不会被推倒，如果不会，要增加或减少多少重量才能使 l 到 r 都被推倒。多米诺骨牌 $i (l + 1 \leq i \leq r)$ 被推倒的条件是：

$$\sum_{j=l}^{i-1} a_j \geq a_i$$

当然，也必须前一个骨牌倒下这个骨牌才有可能倒下。

思路：

预处理前缀和来优化区间求和。那么骨牌 i 被推倒的条件改写为：

$$pre_{i-1} - pre_{l-1} \geq a_i$$

在上面这个式子中， pre_{l-1} 对于每次询问是一个常数，而 pre_{i-1} 和 a_i 只和下标 i 有关。我们对式子稍作变换：

$$-pre_{l-1} \geq a_i - pre_{i-1}$$

如果预处理 $b_i = a_i - pre_{i-1}$ 那么就变成要保证 $\forall i \in [l + 1, r], -pre_{l-1} \geq b_i$ 。

根据高中解不等式的相关知识，我们只需保证这些 b_i 中的最大值小于 $-pre_{l-1}$ 即可。如果说不满足条件，我们最优的调整方案是增加 a_l 的重量，需要增加的值就是 $\max\{b_i\} - (-pre_{l-1})$ 。

我们需要一个维护区间最大值的数据结构，这里选择ST表。

复杂度 $O(n \cdot \log n + q)$ 。

代码：

```
#include <bits/stdc++.h>
#define endl "\n"
using namespace std;
using ll = long long;
struct ST{
    vector<vector<ll>> st;
    ST (const vector<ll> &arr) {
        int n = arr.size();
        st.assign(n, vector<ll>(__lg(n) + 1));
        for (int i = 0; i < n; i++) {
```

```

        st[i][0] = arr[i];
    }
    for (int j = 1; (1 << j) <= n; j++) {
        for (int i = 0; i + (1 << j) <= n; i++) {
            st[i][j] = max(st[i][j - 1], st[i + (1 << j - 1)][j -
1]);
        }
    }
}

ll query(int l, int r) {
    int j = __lg(r - l + 1);
    return max(st[l][j], st[r - (1 << j) + 1][j]);
}

};

int main() {
    ios::sync_with_stdio(false), cin.tie(nullptr);
    ll n, q;
    cin >> n >> q;
    vector<ll> a(n + 1), pre(n + 1), b(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        pre[i] = pre[i - 1] + a[i];
        b[i] = a[i] - pre[i - 1];
    }
    ST st(b);
    for (int i = 0; i < q; i++) {
        int l, r;
        cin >> l >> r;
        if (l == r) {
            cout << 0 << endl;
            continue;
        }
        ll minb = st.query(l + 1, r);
        cout << max(0LL, minb + pre[l - 1]) << endl;
    }
    return 0;
}

```